Table of Contents

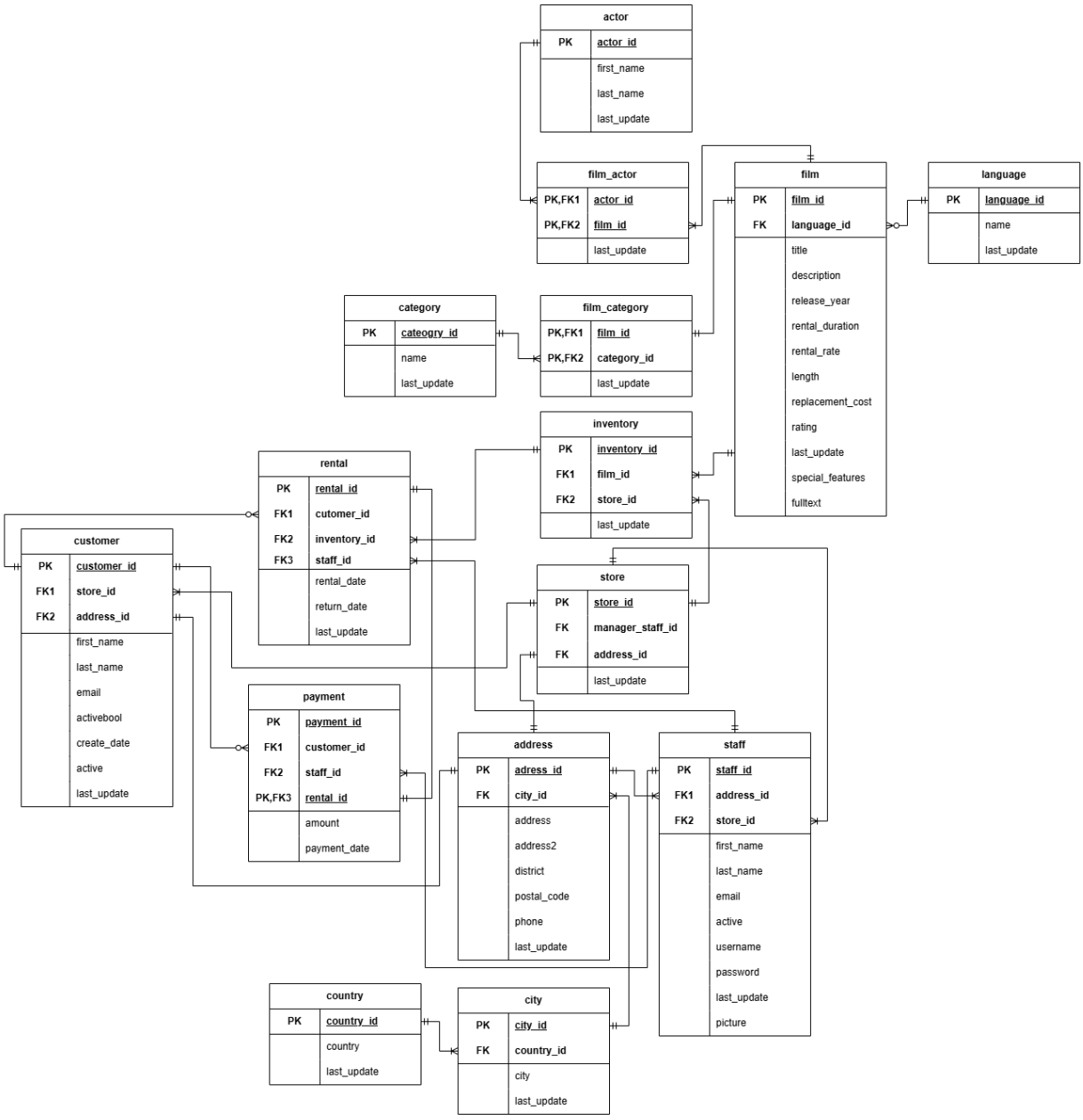
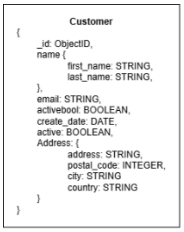


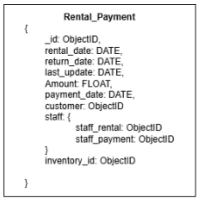
Figure 1 Entity Relationship Diagram of Sakila Database from Assignment 1 as Reference

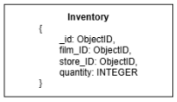
PART A

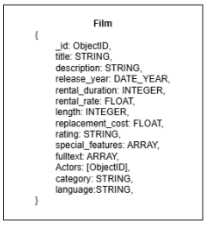
1. Document-Based Data Modeling Representation

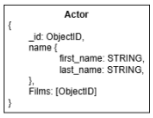
OLTP:

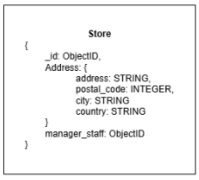

Customer
{
_id: ObjectID,
name {
first_name: STRING,
last_name: STRING,
},
email: STRING,
activebool: BOOLEAN,
create_date: DATE,
active: BOOLEAN,
Address: {
address: STRING,
postal_code: INTEGER,
city: STRING
country: STRING
}
}

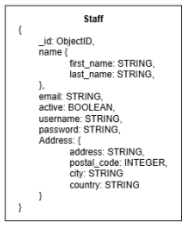

Rental_Payment
{
_id: ObjectID,
rental_date: DATE,
return_date: DATE,
last_update: DATE,
Amount: FLOAT,
payment_date: DATE,
customer: ObjectID
staff: {
staff_rental: ObjectID
staff_payment: ObjectID
}
inventory_id: ObjectID
}


Inventory
{
_id: ObjectID,
film_ID: ObjectID,
store_ID: ObjectID,
quantity: INTEGER
}

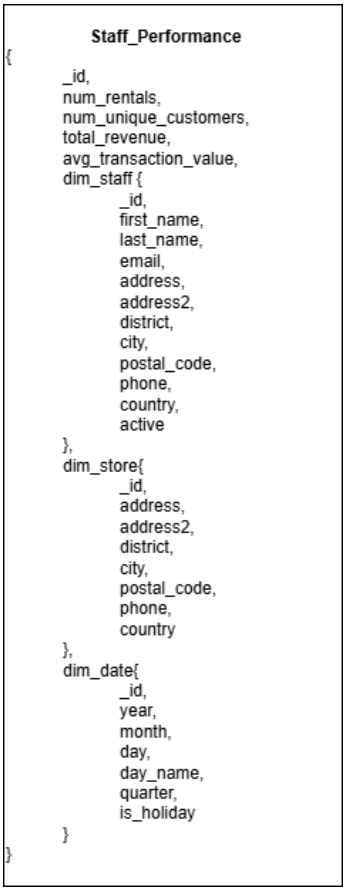

Film
{
_id: ObjectID,
title: STRING,
description: STRING,
release_year: DATE_YEAR,
rental_duration: INTEGER,
rental_rate: FLOAT,
length: INTEGER,
replacement_cost: FLOAT,
rating: STRING,
special_features: ARRAY,
fulltext: ARRAY,
Actors: [ObjectID],
category: STRING,
language:STRING,
}


Actor
{
_id: ObjectID,
name {
first_name: STRING,
last_name: STRING,
},
Films: [ObjectID]
}


Store
{
_id: ObjectID,
Address: {
address: STRING,
postal_code: INTEGER,
city: STRING
country: STRING
}
manager_staff: ObjectID
}


Staff
{
_id: ObjectID,
name {
first_name: STRING,
last_name: STRING,
},
email: STRING,
active: BOOLEAN,
username: STRING,
password: STRING,
Address: {
address: STRING,
postal_code: INTEGER,
city: STRING
country: STRING
}
}

OLAP:


Staff_Performance
{
_id,
num_rentals,
num_unique_customers,
total_revenue,
avg_transaction_value,
dim_staff {
_id,
first_name,
last_name,
email,
address,
address2,
district,
city,
postal_code,
phone,
country,
active
},
dim_store{
_id,
address,
address2,
district,
city,
postal_code,
phone,
country
},
dim_date{
_id,
year,
month,
day,
day_name,
quarter,
is_holiday
}
}

2. Data Dictionary

OLTP:

Collections Name	Object Name	Data Type	Unique
Customer	_id	Object ID	True
	first_name	String	
	last_name	String	
	email	String	
	activebool	Boolean	
	create_date	Date	
	active	Boolean	
Rental_Payment	_id	Object ID	True
	rental_date	Date	
	return_date	Date	
	last_update	Date	
	amount	Float	
	payment_date	Date	
	customer	Object ID	
	staff	Object ID	
	inventory	Object ID	
Inventory	_id	Object ID	True
	film_id	Object ID	
	store_id	Object ID	
	quantity	Integer	
Film	_id	Object ID	True
	title	String	
	description	String	
	release_year	Date	
	rental_duration	Integer	
	rental_rate	Float	
	length	Integer	
	replacement_cost	Float	
	rating	String	
	special_features	Arrays	
	fulltext	Array	
	actors	Object	
	category	String	
	language	String	
Actor	_id	Object ID	True
	first_name	String	
	last_name	String	

	films	Object	
Store	_id	Object ID	True
	manager_staff	Object ID	
Staff	_id	Object ID	True
	first_name	String	
	last_name	String	
	email	String	
	active	Boolean	
	username	String	
	password	String	
Address (embedded into Staff, Store, Customer)	address	String	
	postal_code	Integer	
	city	String	
	country	String	

OLAP:

Collections Name	**Object Name**	**Data Type**	**Unique**
Staff_Performance	_id	Object ID	True
	Num_rentals	Integer	
	Num_unique_customers	Integer	
	Total_Revenue	Float	
	Avg_transaction_value	Float	
Dim_staff (embedded document under staff_performance)	_id	Object ID	
	First_name	String	
	Last_name	String	
	email	String	
	address	String	
	address2	String	
	district	String	
	city	String	
	Postal_code	Integer	
	phone	String	
	country	String	
	active	Boolean	
Dim_store	_id	Object ID	
	address	String	
	address2	String	
	district	String	
	city	String	
	Postal_code	Integer	

	phone	String	
	country	String	
Dim_date	_id	Date	True
	year	Integer	
	month	Integer	
	day	Integer	
	Day_name	String	
	quarter	Integer	
	Is_holiday	Boolean	

3. Data Transformation and Import

Instead of manually creating a database in MongoDB and populating it with sample data, our group leveraged the relational data from the previous assignment using the Sakila database on PostgreSQL (hosted on OpenGauss). We transformed this data into JSON format and then imported it into MongoDB for further processing and use.

3.1 Steps to Migrate & Transform Data from PostgreSQL to JSON for MongoDB Import

Step 1: Create and Associate Aggregated Inventory Data with Rental Records

An aggregated_inventory table was created to group the inventory by film_id and store_id, counting the occurrences for each combination. A new column, aggregated_inventory_id, was then added to the rental table to associate rental records with the aggregated inventory data. The rental table was updated by populating the aggregated_inventory_id based on a join between the inventory and aggregated_inventory tables.

```
CREATE TABLE aggregated_inventory AS
SELECT
    ROW_NUMBER() OVER () AS aggregated_inventory_id,
    film_id,
    store_id,
    COUNT(*) AS count
FROM inventory
GROUP BY film_id, store_id;
```

```
ALTER TABLE rental ADD COLUMN aggregated_inventory_id INT;
```

```
UPDATE rental
```

```
SET aggregated_inventory_id = ai.aggregated_inventory_id
FROM inventory i
JOIN aggregated_inventory ai
ON i.film_id = ai.film_id AND i.store_id = ai.store_id
WHERE rental.inventory_id = i.inventory_id;
```

Step 2: Export Aggregated Inventory Data to JSON

The COPY command was then used to export the aggregated inventory data as JSON objects into a file named inventory.json.

```
COPY (
SELECT
    json_build_object(
        '_id',  ai.aggregated_inventory_id,
        'store',  ai.store_id,
        'film',  ai.film_id,
        'quantity',  ai.count
        )
FROM aggregated_inventory ai
) TO '/home/omm/inventory.json';
```

Step 3: Export Rental and Payment Data to JSON

Relevant fields from payment, rental, and customer tables were then selected and transformed them into JSON objects, and exported them into rental_payment.json.

```
COPY (
SELECT
    json_build_object(
        '_id', p.payment_id,
        'rental_date', r.rental_date,
        'return_date', r.return_date,
        'last_update', r.last_update,
        'amount', p.amount,
        'payment_date', p.payment_date,
        'customer', c.customer_id,
        'staff', json_build_object(
            'staff_rental', r.staff_id,
```

```
            'staff_payment', p.staff_id
        ),
        'inventory', r.aggregated_inventory_id
    )
FROM Payment p
LEFT JOIN Rental r ON p.rental_id = r.rental_id
LEFT JOIN Customer c ON c.customer_id = p.customer_id
) TO '/home/omm/rental_payment.json';
```

Step 4: Export Film Data to JSON

The film, category, language, and actor tables were then queried to create detailed film information in JSON format, then exported it to film.json.

```
COPY (
SELECT
    json_build_object(
        '_id', f.film_id,
        'title', f.title,
        'description', f.description,
        'release_year', f.release_year,
        'rental_duration', f.rental_duration,
        'rental_rate', f.rental_rate,
        'replacement_cost', f.replacement_cost,
        'rating', f.rating,
        'special_features', to_json(f.special_features),
        'fulltext', (
            SELECT json_object_agg(word, frequency)
            FROM (
                SELECT
                    split_part(pair, ':', 1) AS word,
                    CASE
                        WHEN split_part(pair, ':', 2) ~ '^[0-9]+$' THEN
split_part(pair, ':', 2)::int
                        ELSE NULL
                    END AS frequency
                FROM regexp_split_to_table(f.fulltext::text, ' ') AS pair
            ) AS parsed
            WHERE frequency IS NOT NULL
        ),
        'category', c.name,
        'language', TRIM(l.name),
        'actors', (
            SELECT array_agg(a.actor_id)
            FROM actor a
            INNER JOIN film_actor fa ON fa.actor_id = a.actor_id
            WHERE fa.film_id = f.film_id
        )
    )
FROM Film f
LEFT JOIN film_category fc ON fc.film_id = f.film_id
```

```
LEFT JOIN category c ON c.category_id = fc.category_id
LEFT JOIN language l ON l.language_id = f.language_id
) TO '/home/omm/film.json';
```

Step 5: Export Store Data to JSON

Store-related information such as the address was selected and then exported as JSON objects to store.json.

```
COPY (
SELECT
    json_build_object(
        '_id', s.store_id,
        'Address', json_build_object(
            'address', a.address,
            'city', c.city,
            'country', co.country,
            'postal_code', a.postal_code
        ),
        'manager_staff', s.manager_staff_id
    )
FROM store s
LEFT JOIN address a ON s.address_id = a.address_id
LEFT JOIN city c ON a.city_id = c.city_id
LEFT JOIN country co ON c.country_id = co.country_id
) TO '/home/omm/store.json';
```

Step 6: Export Customer Data to JSON

Customer details, including name, address, and email were selected and exported to customer.json.

```
COPY (
SELECT
    json_build_object(
        '_id', cu.customer_id,
        'name', json_build_object(
            'first_name', cu.first_name,
            'last_name', cu.last_name),
        'email', cu.email,
        'active', cu.activebool,
        'create_date', cu.create_date,
        'address', json_build_object(
            'address_line', a.address,
            'city', c.city,
            'country', co.country,
            'postal_code', a.postal_code
        )
   )
FROM customer cu
```

```
LEFT JOIN address a ON cu.address_id = a.address_id
LEFT JOIN city c ON a.city_id = c.city_id
LEFT JOIN country co ON c.country_id = co.country_id
) TO '/home/omm/customer.json;
```

Step 7: Export Staff Data to JSON

Staff data, including address and store details, was selected and exported into staff.json.

```
COPY (
SELECT
    json_build_object(
        '_id', s.staff_id,
        'name', json_build_object(
            'first_name', s.first_name,
            'last_name', s.last_name),
       'store', s.store_id,
        'email', s.email,
        'active', s.active,
       'username', s.username,
       'password', s.password,
        'address', json_build_object(
            'address_line', a.address,
            'city', c.city,
            'country', co.country,
            'postal_code', a.postal_code
        )
  )
FROM staff s
LEFT JOIN address a ON s.address_id = a.address_id
LEFT JOIN city c ON a.city_id = c.city_id
LEFT JOIN country co ON c.country_id = co.country_id
) TO '/home/omm/staff.json';
```

Step 8: Export Actor Data to JSON

Actor and film_actor tables were queried to get the actor details and the films they acted in were then exported the data to actor.json.

```
COPY (
SELECT A
    json_build_object(
        '_id', a.actor_id,
        'name', json_build_object(
            'first_name', a.first_name,
            'last_name', a.last_name),
            'films',  (

            SELECT array_agg(f.film_id)

            FROM film f
```

```
            INNER JOIN film_actor fa ON fa.film_id = f.film_id

            WHERE fa.actor_id = a.actor_id
        )
)
FROM actor a
) TO '/home/omm/actor.json';
```

Step 9: Export Staff Performance Data to JSON

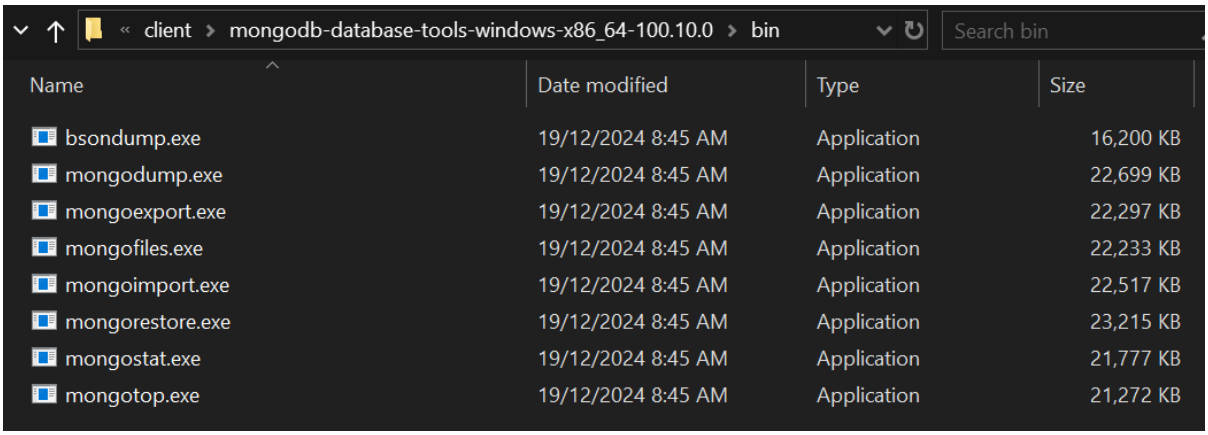
Finally, the staff performance data, including associated staff, store, and date dimensions, was exported to fact_staff_performance.json.

```
COPY (
    SELECT
        json_build_object(
            '_id', ROW_NUMBER() OVER (),
            'num_rentals', fsp.num_rentals,
            'num_unique_customers', fsp.num_unique_customers,
            'total_revenue', fsp.total_revenue,
            'avg_transaction_value', fsp.avg_transaction_value,
            'dim_staff', json_build_object(
                '_id', sf.staff_id,
                'first_name', sf.first_name,
                'last_name', sf.last_name,
                'email', sf.email,
                'address', sf.address,
                'address2', sf.address2,
                'district', sf.district,
                'city', sf.city,
                'postal_code', sf.postal_code,
                'phone', sf.phone,
                'country', sf.country,
                'active', sf.active
            ),
            'dim_store', json_build_object(
                '_id', st.store_id,
                'address', st.address,
                'address2', st.address2,
                'district', st.district,
                'city', st.city,
                'postal_code', st.postal_code,
                'phone', st.phone,
                'country', st.country
            ),
            'dim_date', json_build_object(
                '_id', d.date_id,
                'year', d.year,
                'month', d.month,
                'day', d.day,
                'day_name', d.day_name,
                'quarter', d.quarter,
                'is_holiday', d.is_holiday
            )
```

```
        )
    FROM fact_staff_performance fsp
    LEFT JOIN dim_date d ON fsp.date_id = d.date_id
    LEFT JOIN dim_staff sf ON fsp.staff_id = sf.staff_id
    LEFT JOIN dim_store st ON fsp.store_id = st.store_id
) TO '/home/omm/fact_staff_performance.json';
```

3.2 Steps taken to import JSON files into MongoDB

Step 1: In a new command prompt window, CD to where mongoimport.exe is stored




```
C:\kt\og>cd C:\mongodb8\client\mongodb-database-tools-windows-x86_64-100.10.0\bin
```

Step 2: Run the following commands in the CLI

Each command is structured to specify the database (--db dvd), the collection to import into (--collection <collection_name>), and the file path of the JSON data to be imported (--file <path_to_file>). The mongoimport.exe tool is used to import data from JSON files into MongoDB.

Note: Make sure the MongoDB server is running beforehand. Refer to MongoDBv8 guide.

```
mongoimport.exe --db dvd --collection inventory --file C:\kt\og\inventory.json
mongoimport.exe --db dvd --collection actor --file C:\kt\og\actor.json
mongoimport.exe --db dvd --collection customer --file C:\kt\og\customer.json
mongoimport.exe --db dvd --collection film --file C:\kt\og\film.json
mongoimport.exe --db dvd --collection rental_payment --file
C:\kt\og\rental_payment.json
mongoimport.exe --db dvd --collection staff --file C:\kt\og\staff.json
mongoimport.exe --db dvd --collection store --file C:\kt\og\store.json
mongoimport.exe --db dvd --collection fact_staff_performance --file
C:\kt\og\fact_staff_performance.json
```

```
D:\>cd D:\bin\mongodb8\client\mongodb-database-tools-windows-x86_64-100.10.0\bin

D:\bin\mongodb8\client\mongodb-database-tools-windows-x86_64-100.10.0\bin>mongoimport.exe --db dvd --collection inventory --file C:\kt\og\inventory.json
2025-01-21T21:02:15.192+0800    connected to: mongodb://localhost/
2025-01-21T21:02:15.295+0800    1521 document(s) imported successfully. 0 document(s) failed to import.

D:\bin\mongodb8\client\mongodb-database-tools-windows-x86_64-100.10.0\bin>
D:\bin\mongodb8\client\mongodb-database-tools-windows-x86_64-100.10.0\bin>mongoimport.exe --db dvd --collection actor --file C:\kt\og\actor.json
2025-01-21T21:02:15.362+0800    connected to: mongodb://localhost/
2025-01-21T21:02:15.421+0800    200 document(s) imported successfully. 0 document(s) failed to import.

D:\bin\mongodb8\client\mongodb-database-tools-windows-x86_64-100.10.0\bin>
D:\bin\mongodb8\client\mongodb-database-tools-windows-x86_64-100.10.0\bin>mongoimport.exe --db dvd --collection customer --file C:\kt\og\customer.json
2025-01-21T21:02:15.491+0800    connected to: mongodb://localhost/
2025-01-21T21:02:15.556+0800    600 document(s) imported successfully. 0 document(s) failed to import.

D:\bin\mongodb8\client\mongodb-database-tools-windows-x86_64-100.10.0\bin>
D:\bin\mongodb8\client\mongodb-database-tools-windows-x86_64-100.10.0\bin>mongoimport.exe --db dvd --collection film --file C:\kt\og\film.json
2025-01-21T21:02:15.629+0800    connected to: mongodb://localhost/
2025-01-21T21:02:15.722+0800    1000 document(s) imported successfully. 0 document(s) failed to import.

D:\bin\mongodb8\client\mongodb-database-tools-windows-x86_64-100.10.0\bin>
D:\bin\mongodb8\client\mongodb-database-tools-windows-x86_64-100.10.0\bin>mongoimport.exe --db dvd --collection rental_payment --file C:\kt\og\rental_payment.json
2025-01-21T21:02:15.796+0800    connected to: mongodb://localhost/
2025-01-21T21:02:16.658+0800    33286 document(s) imported successfully. 0 document(s) failed to import.

D:\bin\mongodb8\client\mongodb-database-tools-windows-x86_64-100.10.0\bin>
D:\bin\mongodb8\client\mongodb-database-tools-windows-x86_64-100.10.0\bin>mongoimport.exe --db dvd --collection staff --file C:\kt\og\staff.json
2025-01-21T21:02:16.728+0800    connected to: mongodb://localhost/
2025-01-21T21:02:16.785+0800    55 document(s) imported successfully. 0 document(s) failed to import.
```

```
D:\bin\mongodb8\client\mongodb-database-tools-windows-x86_64-100.10.0\bin>mongoimport.exe --db dvd --collection store --file C:\kt\og\store.json
2025-01-21T21:04:06.261+0800    connected to: mongodb://localhost/
2025-01-21T21:04:06.314+0800    7 document(s) imported successfully. 0 document(s) failed to import.
```

Step 3: Use the mongosh shell to connect to your MongoDB instance and ensure that all collections have been properly imported.

Use the find() method to preview a few documents from each collection, OR use countDocuments() method to check the number of documents in each collection. The number of documents should reflect the number of successful document import listed in the previous step.

```
use dvd
db.inventory.find()
db.actor.find()
db.customer.find()
db.film.find()
db.rental_payment.find()
db.staff.find()
db.store.find()
```

```
use dvd
db.inventory.countDocuments()
db.actor.countDocuments()
db.customer.countDocuments()
db.film.countDocuments()
db.rental_payment.countDocuments()
db.staff.countDocuments()
db.store.countDocuments()
```

4. NoSQL Commands Construction

4.1 One query with Logical operation (AND, OR, NOT)

List down all of rental_payments where the staff making rental and payment are both the same, where either the staff or customer are active and who have not rented an R rated film. In this query, we have utilised the OR and NOT (equal) operator.

```
db.rental_payment.aggregate([
  // Lookup to join the `staff` collection for staff details
  {
    $lookup: {
      from: "staff",
      localField: "staff.staff_rental",
      foreignField: "_id",
      as: "staff_details"
    }
  },
  // Lookup to join the `customer` collection for customer details
  {
    $lookup: {
      from: "customer",
      localField: "customer",
      foreignField: "_id",
      as: "customer_details"
    }
  },
  // Lookup to join the `inventory` and `film` collections for film details
  {
    $lookup: {
      from: "inventory",
      localField: "inventory",
      foreignField: "_id",
      as: "inventory_details"
    }
  },
  {
    $lookup: {
      from: "film",
      localField: "inventory_details.film",
      foreignField: "_id",
      as: "film_details"
    }
  },
  // Match conditions
  {
    $match: {
```

```
      // Staff making rental and payment are the same
      $expr: { $eq: ["$staff.staff_rental", "$staff.staff_payment"] },
      // Either staff or customer is active
      $or: [
        { "staff_details.active": true },
        { "customer_details.active": true }
      ],
      // Exclude rentals for R rated films
      "film_details.rating": { $ne: "R" }
    }
  },
  // Project the necessary fields for output
  {
    $project: {
      _id: 1,
      rental_date: 1,
      return_date: 1,
      payment_date: 1,
      amount: 1,
      customer: "$customer_details.name",
      staff: "$staff_details.name",
      film: "$film_details.title",
      rating: "$film_details.rating"
    }
  }
]);
```

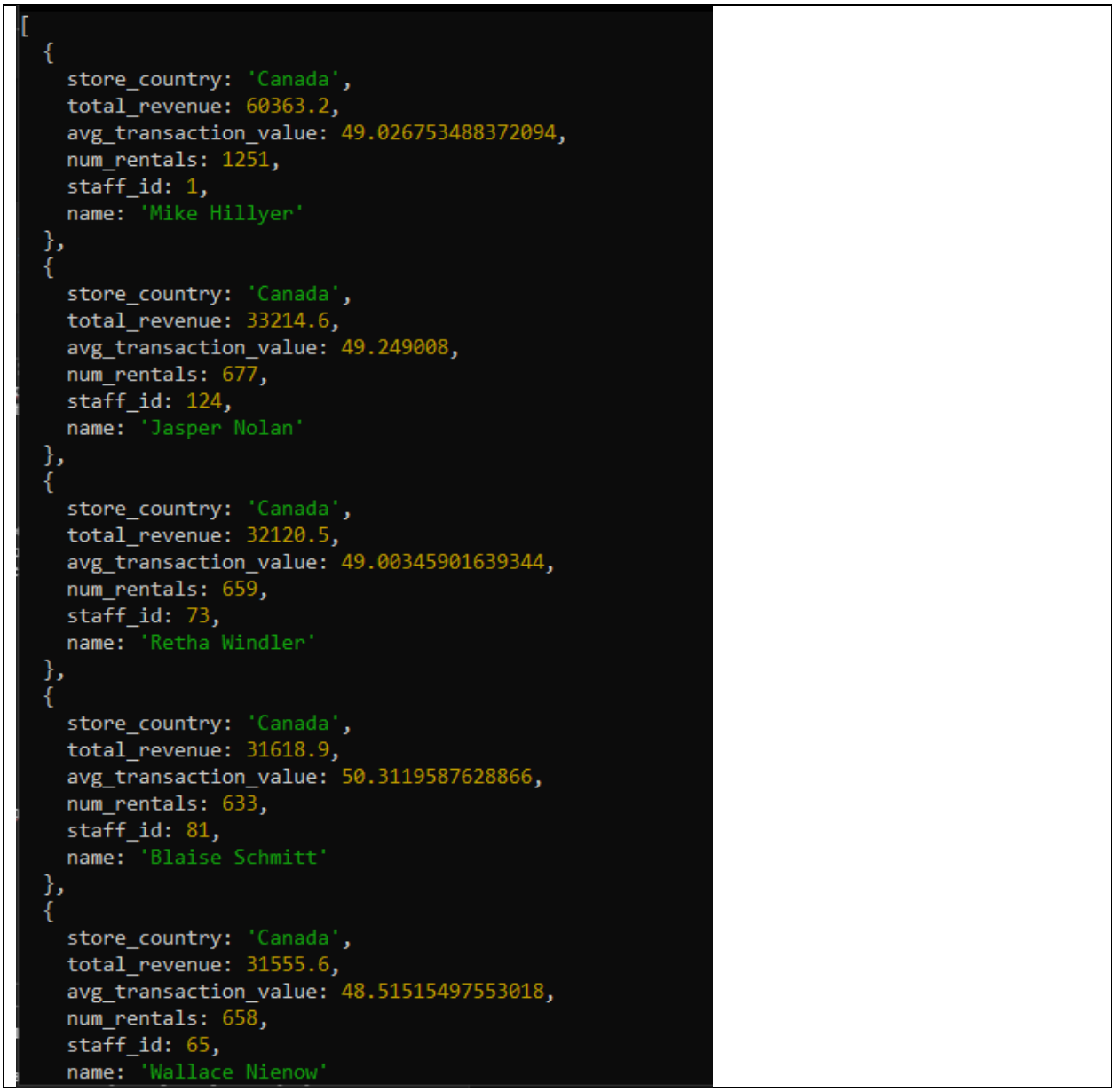
```
[
  {
    _id: 42570,
    rental_date: '2011-02-06 05:31:58',
    return_date: '2011-02-08 04:12:07',
    amount: 60.7,
    payment_date: '2011-02-08 04:12:07',
    customer: [ { first_name: 'PATSY', last_name: 'DAVIDSON' } ],
    staff: [ { first_name: 'Alford', last_name: 'Breitenberg' } ],
    film: [ 'Sleeping Suspects' ],
    rating: [ 'PG-13' ]
  },
  {
    _id: 73297,
    rental_date: '2020-12-25 20:24:40',
    return_date: '2020-12-28 18:14:24',
    amount: 99.5,
    payment_date: '2020-12-28 18:14:24',
    customer: [ { first_name: 'ERICA', last_name: 'MATTHEWS' } ],
    staff: [ { first_name: 'Mollie', last_name: 'Johnson' } ],
    film: [ 'Sleeping Suspects' ],
    rating: [ 'PG-13' ]
  },
  {
    _id: 55794,
    rental_date: '2016-09-11 09:02:29',
    return_date: '2016-09-16 07:19:21',
    amount: 18.1,
    payment_date: '2016-09-16 07:19:21',
    customer: [ { first_name: 'WANDA', last_name: 'PATTERSON' } ],
    staff: [ { first_name: 'Green', last_name: 'Runolfsdottir' } ],
    film: [ 'Pianist Outfield' ],
    rating: [ 'NC-17' ]
  },
```


4.2 One query with Comparison operator (greater than, greater than and equal, less than, etc).

Identify staff members who are top performers that have, as of now, generated $1000 in revenue, have an average transaction value greater than $30 and who works in stores not in the United States. In this query, we utilised the greater than operator.

```
db.fact_staff_performance.aggregate([
  // Group by staff and calculate aggregate metrics
  {
    $group: {
```

```
      _id: "$dim_staff._id",
      first_name: { $first: "$dim_staff.first_name" },
      last_name: { $first: "$dim_staff.last_name" },
      store_country: { $first: "$dim_store.country" },
      total_revenue: { $sum: "$total_revenue" }, // Sum revenue
      avg_transaction_value: { $avg: "$avg_transaction_value" }, // Avg transaction value
      num_rentals: { $sum: "$num_rentals" } // Total rentals
    }
  },
  // Filter the grouped results based on conditions
  {
    $match: {
      total_revenue: { $gt: 1000 }, // Revenue exceeds $1000
      avg_transaction_value: { $gt: 30 }, // Avg transaction value greater than $30
      store_country: { $ne: "United States" } // Exclude US-based stores
    }
  },
  // Sort by total revenue in descending order
  {
    $sort: { total_revenue: -1 }
  },
  // Project only necessary fields for output
  {
    $project: {
      _id: 0,
      staff_id: "$_id",
      name: { $concat: ["$first_name", " ", "$last_name"] },
      store_country: 1,
      total_revenue: 1,
      avg_transaction_value: 1,
      num_rentals: 1
    }
  }
]);
```

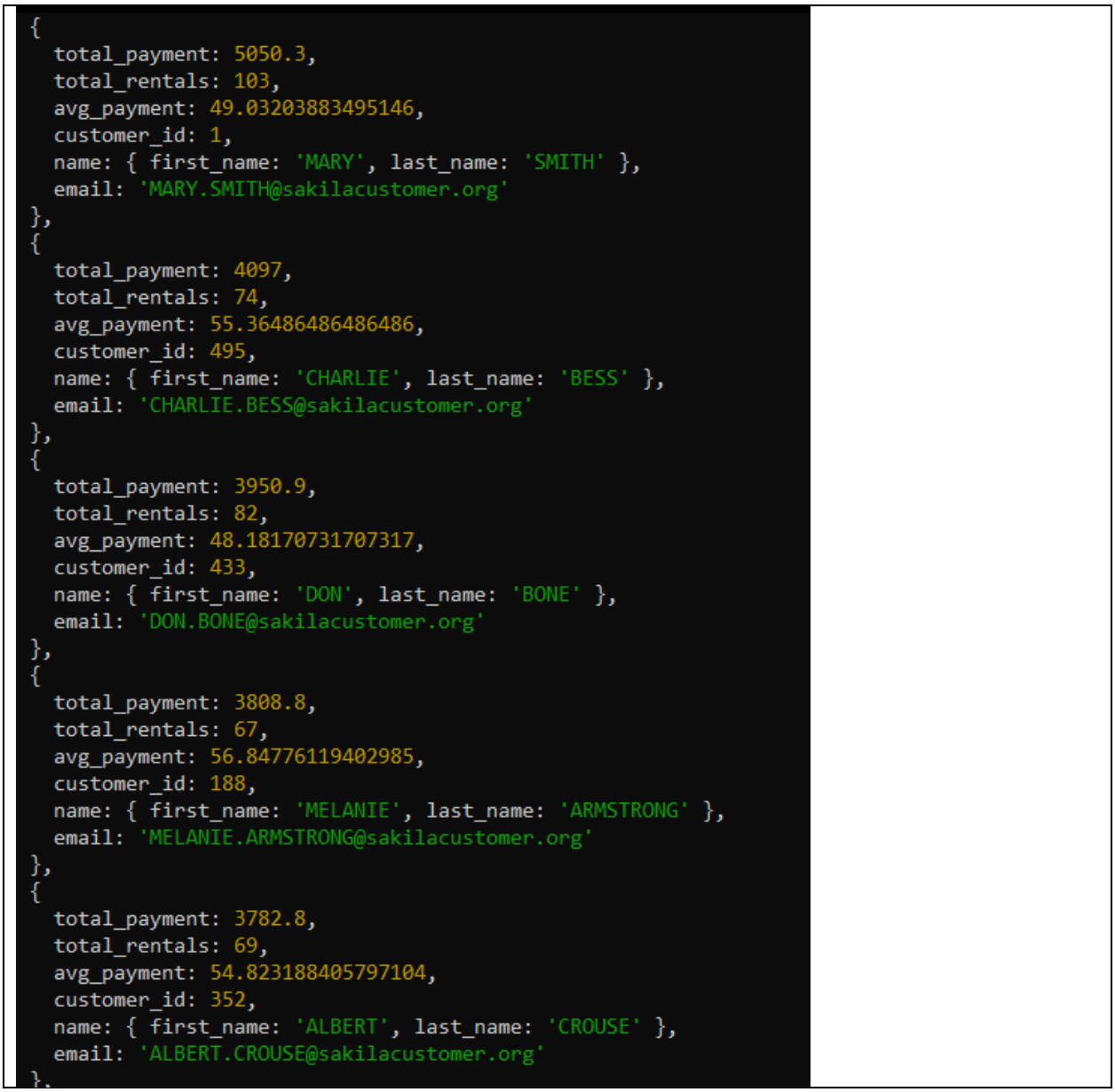


4.3 Aggregate function (sum, average, max, min, count)

Identify the top customers based on the total rental amount they have paid. Specifically, by computing the total amount paid by each customer (sum), by counting how many rentals each customer has made (count) and by find the customer with the highest total payment amount (max).

```
db.rental_payment.aggregate([
  // Group by customer and calculate total payment and rental count
  {
    $group: {
      _id: "$customer", // Group by customer ID
```

```
    total_payment: { $sum: "$amount" }, // Total amount paid
    total_rentals: { $count: {} }, // Count of rentals
    avg_payment: { $avg: "$amount" } // Average payment per rental
   }
  },
  // Sort by total payment in descending order
  {
   $sort: { total_payment: -1 }
  },
  // Limit to the top 10 customers
  {
   $limit: 10
  },
  // Lookup to get customer details
  {
   $lookup: {
    from: "customer",
    localField: "_id",
    foreignField: "_id",
    as: "customer_details"
   }
  },
  // Project final output
  {
   $project: {
    _id: 0,
    customer_id: "$_id",
    name: { $arrayElemAt: ["$customer_details.name", 0] }, // Customer name
    email: { $arrayElemAt: ["$customer_details.email", 0] }, // Customer email
    total_payment: 1,
    total_rentals: 1,
    avg_payment: 1
   }
  }
]);
```

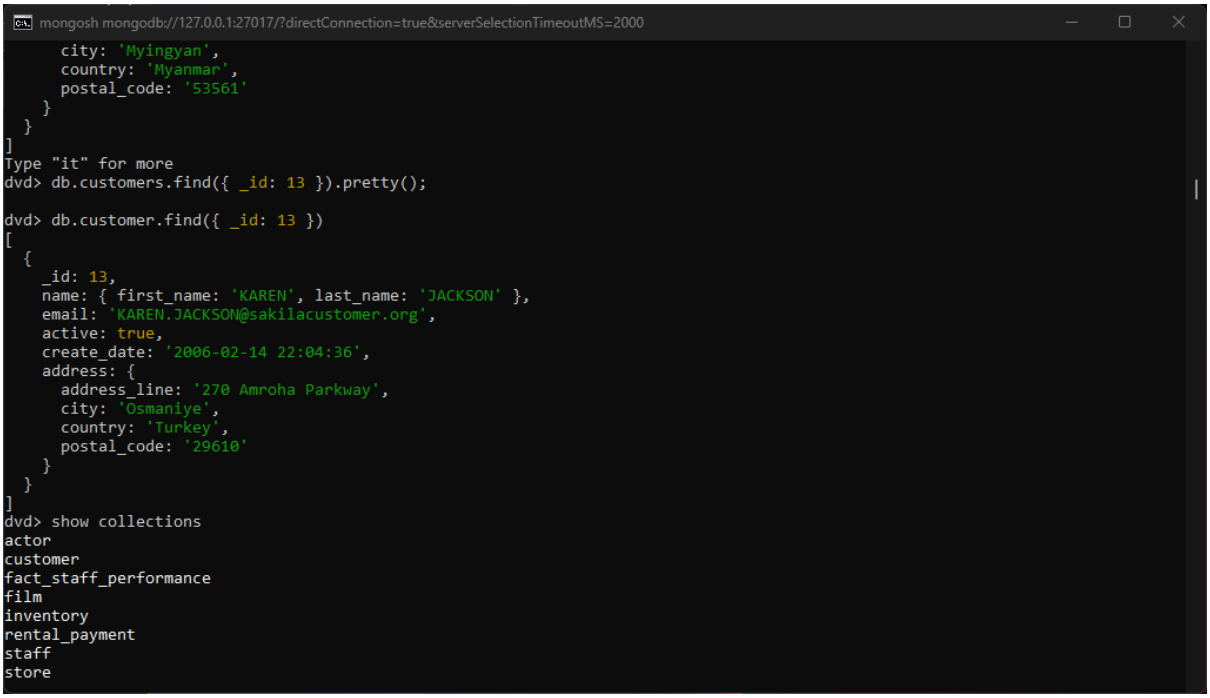

{
total_payment: 5050.3,
total_rentals: 103,
avg_payment: 49.03203883495146,
customer_id: 1,
name: { first_name: 'MARY', last_name: 'SMITH' },
email: 'MARY.SMITH@sakilacustomer.org'
},
{
total_payment: 4097,
total_rentals: 74,
avg_payment: 55.36486486486486,
customer_id: 495,
name: { first_name: 'CHARLIE', last_name: 'BESS' },
email: 'CHARLIE.BESS@sakilacustomer.org'
},
{
total_payment: 3950.9,
total_rentals: 82,
avg_payment: 48.18170731707317,
customer_id: 433,
name: { first_name: 'DON', last_name: 'BONE' },
email: 'DON.BONE@sakilacustomer.org'
},
{
total_payment: 3808.8,
total_rentals: 67,
avg_payment: 56.84776119402985,
customer_id: 188,
name: { first_name: 'MELANIE', last_name: 'ARMSTRONG' },
email: 'MELANIE.ARMSTRONG@sakilacustomer.org'
},
{
total_payment: 3782.8,
total_rentals: 69,
avg_payment: 54.823188405797104,
customer_id: 352,
name: { first_name: 'ALBERT', last_name: 'CROUSE' },
email: 'ALBERT.CROUSE@sakilacustomer.org'
},

4.4 Update record(s) based on certain criteria

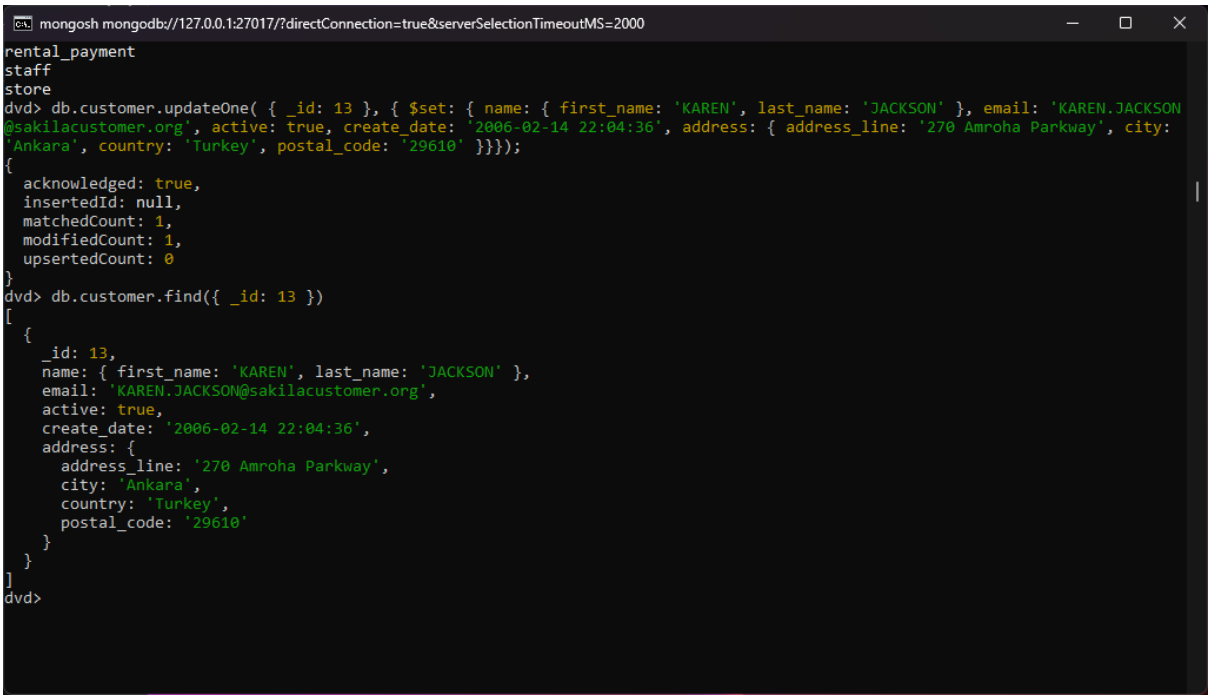
This following command updates a particular record (record 13) which in this case would be the ‘address.city’ from “Osmaniye” to “Ankara”.

```
db.customers.updateOne(
  { _id: 13 },
  {
    $set: {
      name: {
        first_name: 'KAREN',
        last_name: 'JACKSON'
      },
      email: 'KAREN.JACKSON@sakilacustomer.org',
      active: true,
      create_date: '2006-02-14 22:04:36',
      address: {
        address_line: '270 Amroha Parkway',
        city: 'Ankara',
        country: 'Turkey',
        postal_code: '29610'
      }
    }
  }
);
```

Before Query Execution:



After Execution:

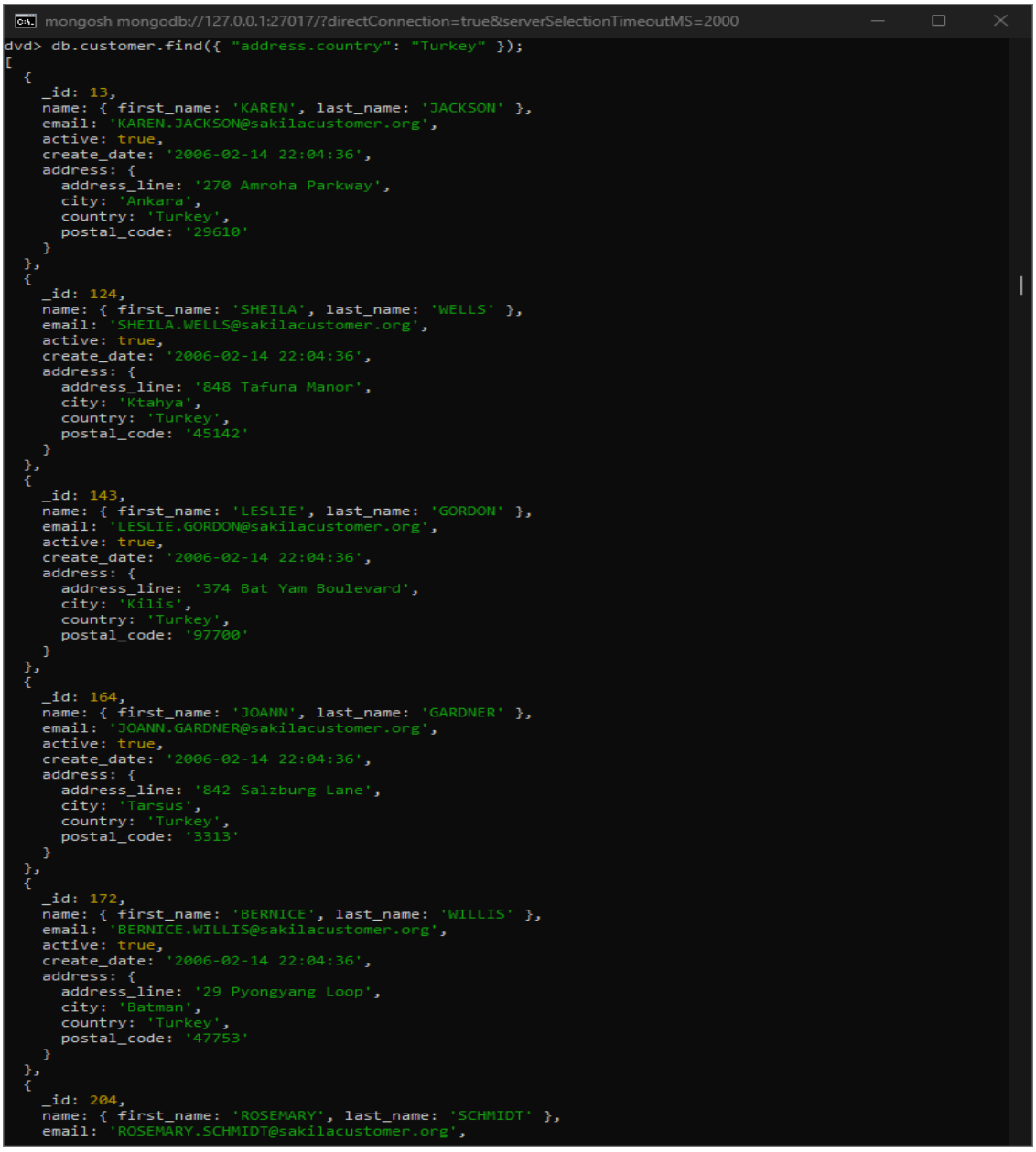


4.5 Delete some specific records based on certain criteria

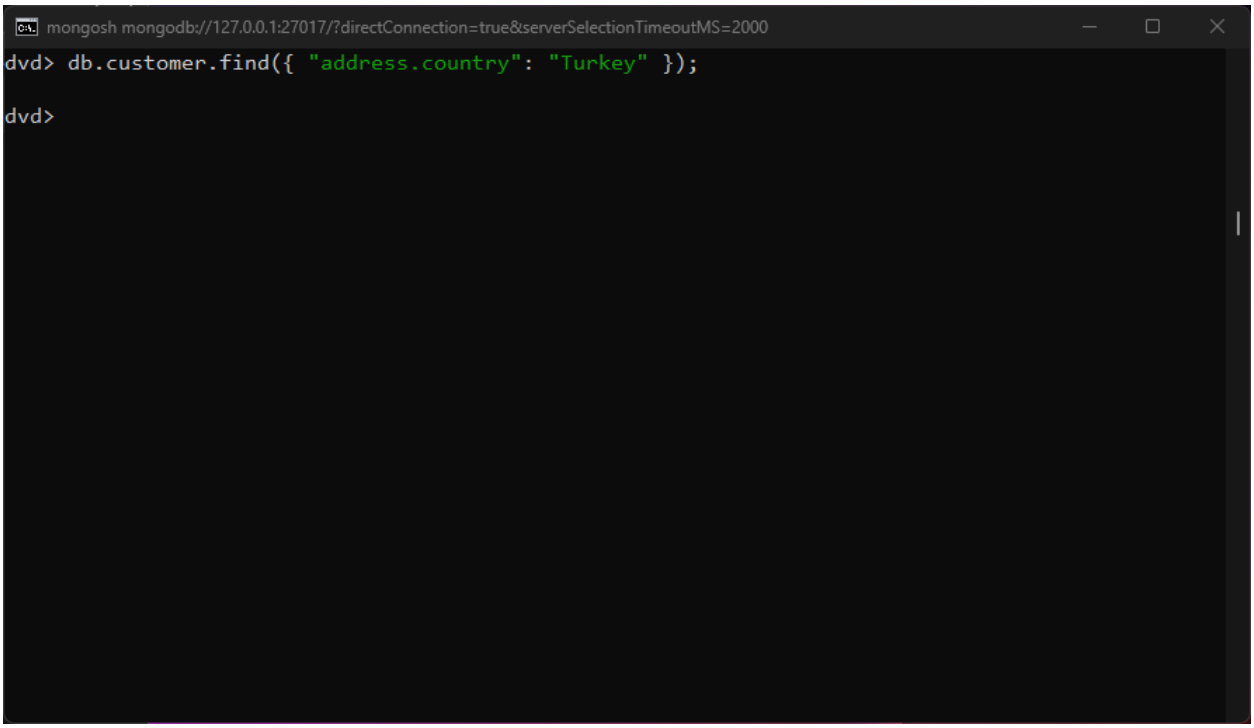
Delete all customers from a specific country (Turkey).

```
db.customers.deleteMany({ "address.country": "Turkey" });
```

Before Query Execution:



After Execution:



Documents with the address.country : “Turkey” cannot be found in the collection. They are successfully deleted.

4.6 Two NoSQL commands that are not covered in lecture

a. createIndex()

The createIndex() command creates an index on the title, category, release_year, and actors fields in the film collection to make queries faster when filtering or sorting by these fields. This explain() command is then used to illustrate the index being used by explaining how MongoDB executes the query, showing details like the query plan, index utilization, and performance metrics (e.g., time taken and documents examined). You can find the use of the index under the **winningPlan s**ection, where IXSCAN and the indexName are mentioned, and the execution time is shown as executionTimeMillis in the **executionStats** section.

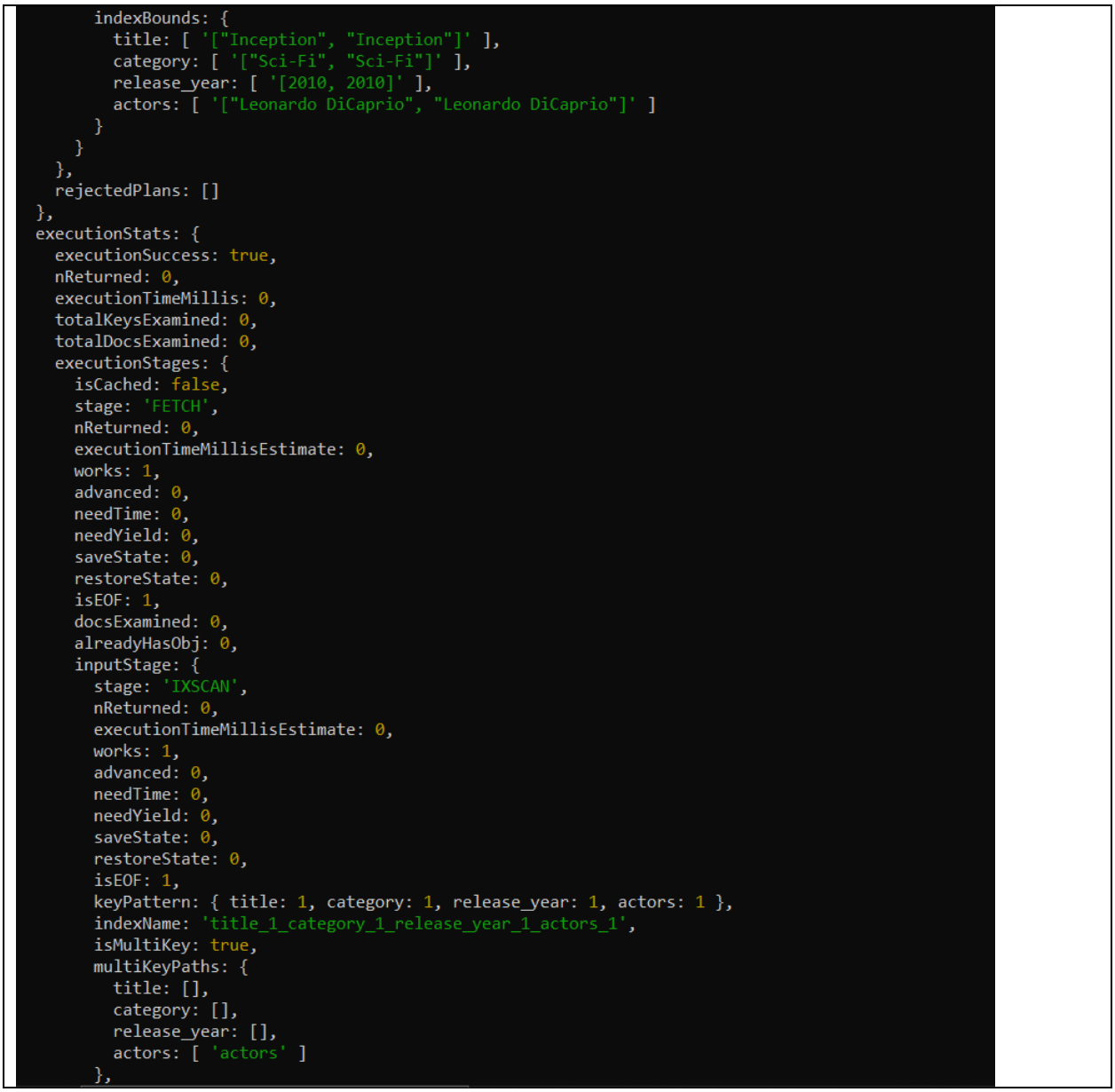
```
db.film.createIndex(
  {
    title: 1,
    category: 1,
    release_year: 1,
    actors: 1
  }
)
```

Command to Illustrate Index is being Used:

```
db.film.find({
  title: "Inception",
  category: "Sci-Fi",
  release_year: 2010,
  actors: "Leonardo DiCaprio"
}).explain("executionStats")
```

Output:

```
dvd> db.film.find({
...   title: "Inception",
...   category: "Sci-Fi",
...   release_year: 2010,
...   actors: "Leonardo DiCaprio"
... }).explain("executionStats")
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'dvd.film',
    parsedQuery: {
      '$and': [
        { actors: { '$eq': 'Leonardo DiCaprio' } },
        { category: { '$eq': 'Sci-Fi' } },
        { release_year: { '$eq': 2010 } },
        { title: { '$eq': 'Inception' } }
      ]
    },
    indexFilterSet: false,
    planCacheShapeHash: 'EDD2230D',
    planCacheKey: '95326296',
    optimizationTimeMillis: 0,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'FETCH',
      inputStage: {
        stage: 'IXSCAN',
        keyPattern: { title: 1, category: 1, release_year: 1, actors: 1 },
        indexName: 'title_1_category_1_release_year_1_actors_1',
        isMultiKey: true,
        multiKeyPaths: {
          title: [],
          category: [],
          release_year: [],
          actors: [ 'actors' ]
        },
        isUnique: false,
        isSparse: false,
        isPartial: false,
        indexVersion: 2,
        direction: 'forward',
```


indexBounds: {
title: ['["Inception", "Inception"]'],
category: ['["Sci-Fi", "Sci-Fi"]'],
release_year: ['[2010, 2010]'],
actors: ['["Leonardo DiCaprio", "Leonardo DiCaprio"]']
}
}
},
rejectedPlans: []
},
executionStats: {
executionSuccess: true,
nReturned: 0,
executionTimeMillis: 0,
totalKeysExamined: 0,
totalDocsExamined: 0,
executionStages: {
isCached: false,
stage: 'FETCH',
nReturned: 0,
executionTimeMillisEstimate: 0,
works: 1,
advanced: 0,
needTime: 0,
needYield: 0,
saveState: 0,
restoreState: 0,
isEOF: 1,
docsExamined: 0,
alreadyHasObj: 0,
inputStage: {
stage: 'IXSCAN',
nReturned: 0,
executionTimeMillisEstimate: 0,
works: 1,
advanced: 0,
needTime: 0,
needYield: 0,
saveState: 0,
restoreState: 0,
isEOF: 1,
keyPattern: { title: 1, category: 1, release_year: 1, actors: 1 },
indexName: 'title_1_category_1_release_year_1_actors_1',
isMultiKey: true,
multiKeyPaths: {
title: [],
category: [],
release_year: [],
actors: ['actors']
},

```
        actors: [ 'actors' ]
      },
      isUnique: false,
      isSparse: false,
      isPartial: false,
      indexVersion: 2,
      direction: 'forward',
      indexBounds: {
        title: [ '["Inception", "Inception"]' ],
        category: [ '["Sci-Fi", "Sci-Fi"]' ],
        release_year: [ '[2010, 2010]' ],
        actors: [ '["Leonardo DiCaprio", "Leonardo DiCaprio"]' ]
      },
      keysExamined: 0,
      seeks: 1,
      dupsTested: 0,
      dupsDropped: 0
    }
  }
},
queryShapeHash: 'ED4671DDAFFB1CE6C8A24A9B3A3B2662E088925A5F3814D56EB1FFFE0E586C0C',
command: {
  find: 'film',
  filter: {
    title: 'Inception',
    category: 'Sci-Fi',
    release_year: 2010,
    actors: 'Leonardo DiCaprio'
  },
  '$db': 'dvd'
},
serverInfo: {
  host: 'LAPTOP-2D62T37B',
  port: 27017,
  version: '8.0.4',
  gitVersion: 'bc35ab4305d9920d9d0491c1c9ef9b72383d31f9'
},
serverParameters: {
  internalQueryFacetBufferSizeBytes: 104857600,
  internalQueryFacetMaxOutputDocSizeBytes: 104857600,
  internalLookupStageIntermediateDocumentMaxSizeBytes: 104857600,
  internalDocumentSourceGroupMaxMemoryBytes: 104857600,
  internalQueryMaxBlockingSortMemoryUsageBytes: 104857600,
  internalQueryProhibitBlockingMergeOnMongoS: 0,
  internalQueryMaxAddToSetBytes: 104857600,
  internalDocumentSourceSetWindowFieldsMaxMemoryBytes: 104857600,
  internalQueryFrameworkControl: 'trySbeRestricted',
  internalQueryPlannerIgnoreIndexWithCollationForRegex: 1
},
ok: 1
```


b. validate()

The validate command checks if a collection's data is correct, finding problems like incorrect structure, broken indexes, or corrupted documents. The result of the validate command used shows that the actor collection is in good condition, with no invalid or non-compliant documents, one valid index, and no errors or warnings.

```
db.runCommand({ validate: "actor" })
```

Output:

```
dvd> db.runCommand({ validate: "actor" })
{
  ns: 'dvd.actor',
  uuid: UUID('bf434020-2cc6-4e80-a5f4-ac0ab6338e32'),
  nInvalidDocuments: 0,
  nNonCompliantDocuments: 0,
  nrecords: 200,
  nIndexes: 1,
  keysPerIndex: { _id_: 200 },
  indexDetails: { _id_: { valid: true } },
  valid: true,
  repaired: false,
  warnings: [],
  errors: [],
  extraIndexEntries: [],
  missingIndexEntries: [],
  corruptRecords: [],
  ok: 1
}
```